

KCPerformance

Automatisering rolpoort

Tobias van de Ven
4-6-2025

Inhoud

Inleiding	3
Projectdefinitie	4
Doelstelling.....	4
Scope	4
Planning	5
Manier van plannen	5
Planning bijhouden	5
Ervaring met de planning	5
Architectuur en systeemontwerp	7
Overzicht van het systeem.....	7
Systeemflow	8
Hardwarecomponenten	9
Gebruikte hardware.....	9
Wiring-overzicht.....	10
Waarom deze setup?	10
Software, ontwikkelomgeving en logica	11
Ontwikkelomgeving.....	11
Structuur en werking	11
Web interface	12
Stabiliteit en foutafhandeling	12
Functie diagram	12
Ontwikkeling en Implementatie.....	13
Ontwikkeling van de software	13
Installatiehandleiding	14
Testfase.....	15
Testplan en testmethodologieën prototype.....	15
Testcases en resultaten	16
Bugfixing en kwaliteit.....	16
Conclusie	16
Test foto's	17
Testplan en testmethodologieën eindproduct.....	18

Testcases	18
Conclusie	18
Evaluatie en reflectie	19
Bijlagen	20

Inleiding

Bij KCPerformance wordt de rolpoort gebruikt om auto's de werkplaats in te rijden. Op dit moment kan de poort geopend worden door een aantal afstandsbedieningen, maar die zijn niet ideaal. Ze gaan vaak kapot, raken kwijt, en er zijn er niet genoeg van voor alle medewerkers. Het gevolg is dat werknemers zonder afstandsbediening regelmatig voor een gesloten poort staan zonder zelf de mogelijkheid hebben om deze te openen.

Om dit probleem op te lossen wil ik de rolpoort automatiseren met behulp van een **Raspberry Pi Pico W**. Het idee is dat de Pi wordt aangesloten op de bestaande knoppen voor omhoog, stop en omlaag, en dan vervolgens via een simpele webapp de knoppen worden bediend door medewerkers via een telefoon. Geen afstandsbedieningen meer nodig, gewoon een knop op je scherm.

In dit document beschrijf ik het volledige ontwikkelproces van de probleemstelling en de gekozen hardware/software, tot het schrijven van de code, het bouwen van de interface, het testen van het prototype en de uiteindelijke (mogelijke) implementatie. Daarbij ga ik ook in op de keuzes die ik onderweg maak en hoe ik bepaalde technische uitdagingen aanpak.

Projectdefinitie

Doelstelling

Het doel van dit project is om de rolpoort van KCPerformance te automatiseren met een extra besturingsoptie. Op dit moment is de poort alleen te bedienen met een beperkt aantal afstandsbedieningen, die vaak kapotgaan of kwijtraken. Door een alternatieve oplossing te ontwikkelen, moet het voor meerdere medewerkers mogelijk worden om de poort veilig, eenvoudig en betrouwbaar te openen via een webapp. De bestaande functies zoals het alarmsysteem en de originele fysieke bediening van de poort moeten daarbij volledig intact blijven.

In het kort:

- De rolpoort krijgt een extra digitale besturingsoptie via wifi.
- De bestaande functionaliteiten blijven behouden.
- De oplossing moet veilig, stabiel en gebruiksvriendelijk zijn.

Scope

Dit project richt zich op het ontwerpen, bouwen en testen van een werkend prototype oplossing. De focus ligt op:

- Hardware-integratie van de Raspberry Pi Pico W met poortbesturing.
- Ontwikkelen van een eenvoudige web interface voor bediening (via HTTP requests).
- Documentatie van het hele proces.

Het project bevat géén integratie met externe systemen (zoals alarmsystemen of toegangscontrole), en ook geen Cloud gebaseerde bediening. Alles blijft lokaal.

Planning

Tijdens het project heb ik een duidelijke planning opgesteld om overzicht te houden op de voortgang van mijn examen werkzaamheden. Deze planning heb ik visueel weergegeven in een Gantt-diagram (zie foto onderaan dit hoofdstuk), waarin de verschillende taken, hun duur en status worden weergegeven.

Manier van plannen

Ik heb gekozen voor een eenvoudige Gantt-structuur waarin ik elke taak begin- en einddatum gaf, inclusief visuele kleuren om onderscheid te maken tussen gepland werk, afgeronde taken, wijzigingen in de planning en deadlines. Elke taak kreeg een vooraf ingeschatte duur, waarna ik deze taken logisch over de weken heb verdeeld. De planning is opgedeeld in de volgende fasen:

- **Vorbereiding:** research, diagrammen en softwareontwerp
- **Ontwikkeling:** code schrijven, testen en prototyping
- **Afronding:** controle, implementatie en het schrijven van het examenverslag

In de planning zijn verschillende kleuren gebruikt om de status van taken weer te geven:

- **Grijs:** geplande taken
- **Groen:** afgeronde taken
- **Oranje:** verschuiving in de planning
- **Rood:** deadlines

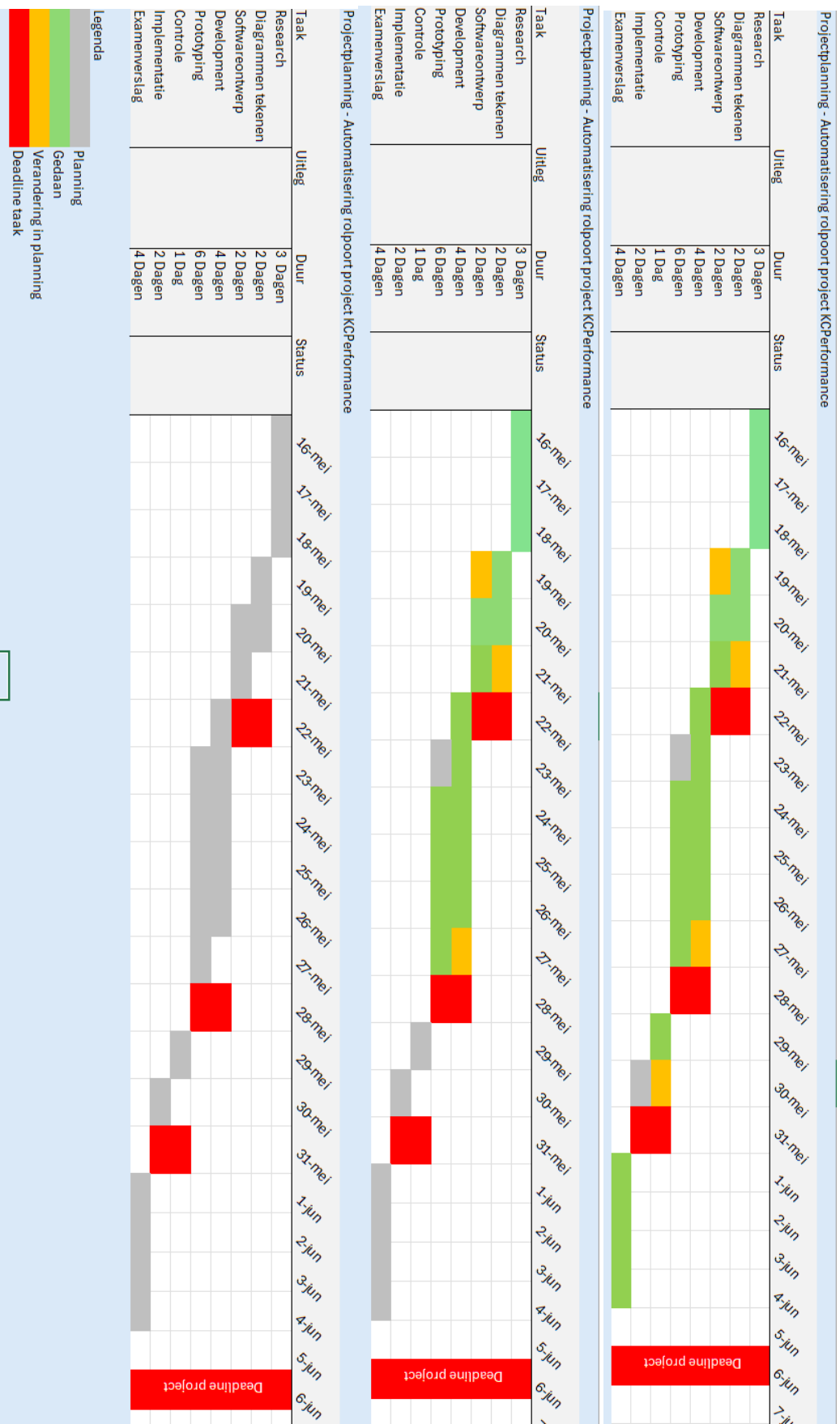
Planning bijhouden

Tijdens het project heb ik actief mijn planning bijgewerkt. Zodra een taak was afgerond, kleurde ik deze groen. Taken die ik iets later of eerder heb uitgevoerd dan gepland, gaf ik een oranje kleur. Deze methode zorgde ervoor dat ik consistent overzicht had over de voortgang en eventuele knelpunten snel kon zien.

Ook wanneer ik taken moest opsplitsen of combineren, paste ik de planning handmatig aan. Hierdoor bleef het overzichtelijk.

Ervaring met de planning

De planning bleek over het algemeen goed te volgen en realistisch. Hoewel er een aantal kleine verschuivingen waren, bijvoorbeeld bij het softwareontwerp en de development fase, bleef het project beheersbaar en binnen de gestelde tijd. De opzet van de planning was simpel, maar effectief genoeg om structuur te bieden. Het gaf rust om steeds te weten wat er nog moest gebeuren en wanneer.



Architectuur en systeemontwerp

In dit hoofdstuk beschrijf ik hoe het systeemtechnisch in elkaar zit: van hardware tot software. Ik leg uit welke keuzes ik heb gemaakt op het gebied van componenten, hoe alles met elkaar verbonden is en hoe de logica van de code werkt. Ook laat ik zien hoe de flow van het systeem eruitziet en kijk ik naar de beperkingen en afwegingen die tijdens het ontwerpen zijn meegenomen.

Overzicht van het systeem

Het systeem bestaat uit een Raspberry Pi Pico W die verbonden is met het bestaande bedieningspaneel van de rolpoort. De Pi stuurt drie relais aan die gelijk staan aan de functies *omhoog*, *stop* en *omlaag* van de poort. In plaats van het fysiek indrukken van de knop wordt dit signaal nu met software gemaakt via een web interface die draait op de Pi zelf.

De gebruiker opent een webpagina op zijn telefoon (binnen het lokale netwerk) en stuurt via die interface een HTTP-request naar de Raspberry Pi. Dit request activeert vervolgens het juiste relais om het signaal naar de poort te simuleren.

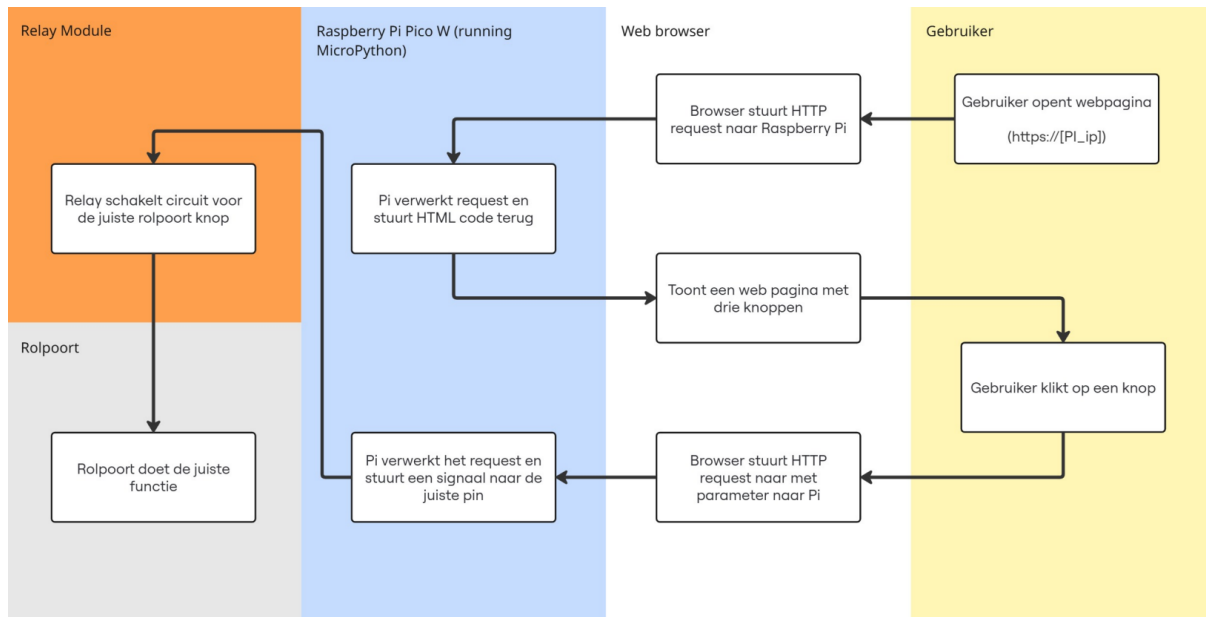
De hoofdelementen van het systeem zijn:

- **Webserver op de Pi Pico W:** Bedient de web interface en verwerkt HTTP-requests.
- **Relaismodule:** Schakelt de fysieke poortbediening.
- **HTML/CSS/JS-interface:** Webpagina die door gebruikers wordt geopend om de poort te bedienen.
- **WiFi Netwerk (lokaal):** Verbind gebruikers met de Pi.

Het systeem werkt volledig lokaal. De Pi hoeft geen toegang te hebben tot het internet, alleen tot het interne WiFi netwerk. Hierdoor blijft het geheel relatief veilig en onafhankelijk van externe servers of diensten.

Systeemflow

Hieronder zie je een flowchart die laat zien hoe het systeem werkt van input tot actie:



Deze opzet zorgt ervoor dat:

- De originele knoppen op het systeem blijven werken
- De afstandsbedieningen niet vervangen hoeven te worden door nieuwe fysieke apparaten
- Medewerkers simpel via hun telefoon de poort kunnen bedienen zolang ze op het netwerk zitten

Hardwarecomponenten

Voor dit project heb ik gekozen voor simpele, betrouwbare en makkelijk verkrijgbare hardwarecomponenten. Omdat het systeem in een werkplaatsomgeving draait, is het belangrijk dat alles robuust, stabiel en eenvoudig te onderhouden is.

Gebruikte hardware

- **Raspberry Pi Pico W**

De centrale microcontroller die zowel de webserver draait als het relais aanstuurt. Deze is gekozen vanwege de ingebouwde WiFi-module, lage stroomverbruik en ondersteuning voor MicroPython.

- **3x 5V relaismodule**

Elk relais simuleert een druk op een van de knoppen van de bestaande poortbediening (omhoog, omlaag en stop). Door het relais 0,5 seconden te activeren wordt de verbinding van de knop rond gemaakt alsof er op een knop wordt gedrukt.

- **Voeding**

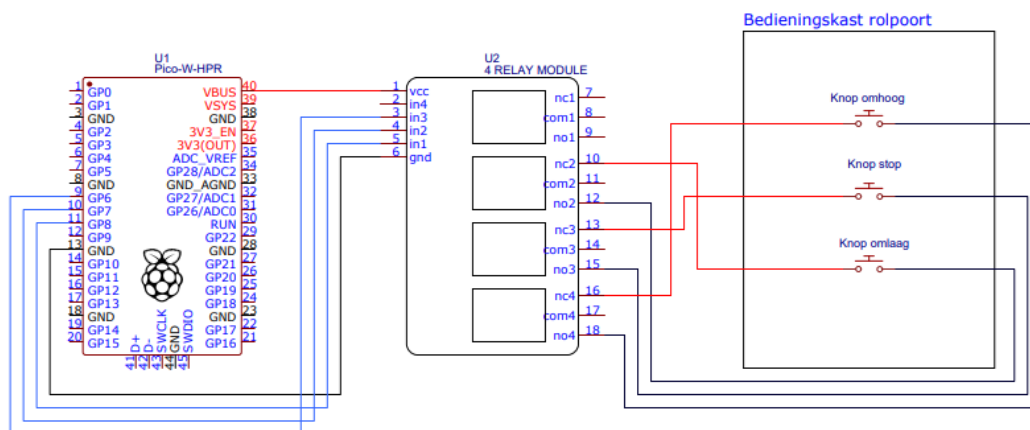
De Raspberry Pi wordt gevoed via een standaard USB-adapter (5V). Het relais worden ook van stroom voorzien via de Pi's vcc.

- **Draden**

Voor het maken van de verbinding tussen de Pi, de relaismodule en de bestaande poortbediening zijn standaard Dupont kabels gebruikt.

Wiring-overzicht

In onderstaande wiring-diagram zie je hoe alles met elkaar verbonden is. De GPIO-pinnen 6, 7 en 8 van de Pico zijn verbonden met de drie relais, die verbonden zijn met de bestaande knoppen.



Waarom deze setup?

- De Raspberry Pi Pico W is krachtig genoeg voor dit soort taken en het werkt met MicroPython wat snel draait en opstart.
- Het relais maken het mogelijk om veilig en gescheiden verbinding te maken met het bestaande systeem.
- Deze setup is relatief goedkoop, goed vervangbaar en kan makkelijk uitgebreid worden als het systeem in de toekomst meer functies krijgt.

Software, ontwikkelomgeving en logica

De volledige besturing van het systeem draait op een **Raspberry Pi Pico W**, die automatisch de code uitvoert zodra deze opstart. De software is geschreven in **MicroPython**, een efficiënte en lichte aftakking van Python voor microcontrollers. De keuze voor MicroPython is bewust gemaakt: het is snel op te zetten, eenvoudig te debuggen en geschikt voor het aansturen van GPIO's en het opzetten van een eenvoudige webserver.

Ontwikkelomgeving

Tijdens de ontwikkeling heb ik de volgende tools gebruikt:

- **Thonny IDE:** een ontwikkelomgeving die goede ondersteuning biedt voor MicroPython en het uploaden van code naar de Pico W.
- **Wifi-omgeving van KCPerformance:** de Pico W verbindt automatisch met het lokale netwerk, zodat de bediening van de rolpoort op elk moment via een telefoon mogelijk is.

Structuur en werking

Alle functionaliteit zit in een bestand: `main.py`. Dit script:

- **Verbindt met het WiFi netwerk**
- **Start een lokale HTTP-server**
- **Reageert op HTTP-requests** voor `/up`, `/stop` en `/down` met bijbehorende relais acties.
- **Toont een eenvoudige web interface** als geen specifieke actie wordt opgevraagd

Het relais zijn aangesloten op GPIO-pins 6, 7 en 8. Bij een actie (zoals `/up`) wordt de juiste pin 0,5 seconden laag gezet om het relais te triggeren, waarna deze weer hoog wordt gezet.

Voorbeeld logica uit de code:

```
85 def gate_up():
86     relay1.value(0)
87     time.sleep(0.5)
88     relay1.value(1)
89
```

Deze aanpak zorgt ervoor dat het relais slechts kort geactiveerd wordt.

Web interface

De HTML-interface is hardcoded in de code zelf en wordt meegegeven als response op het standaard GET-request naar de Pi. De front-end bevat drie knoppen (“UP”, “STOP” en “DOWN”) die via fetch() een GET-request doen aan de server.

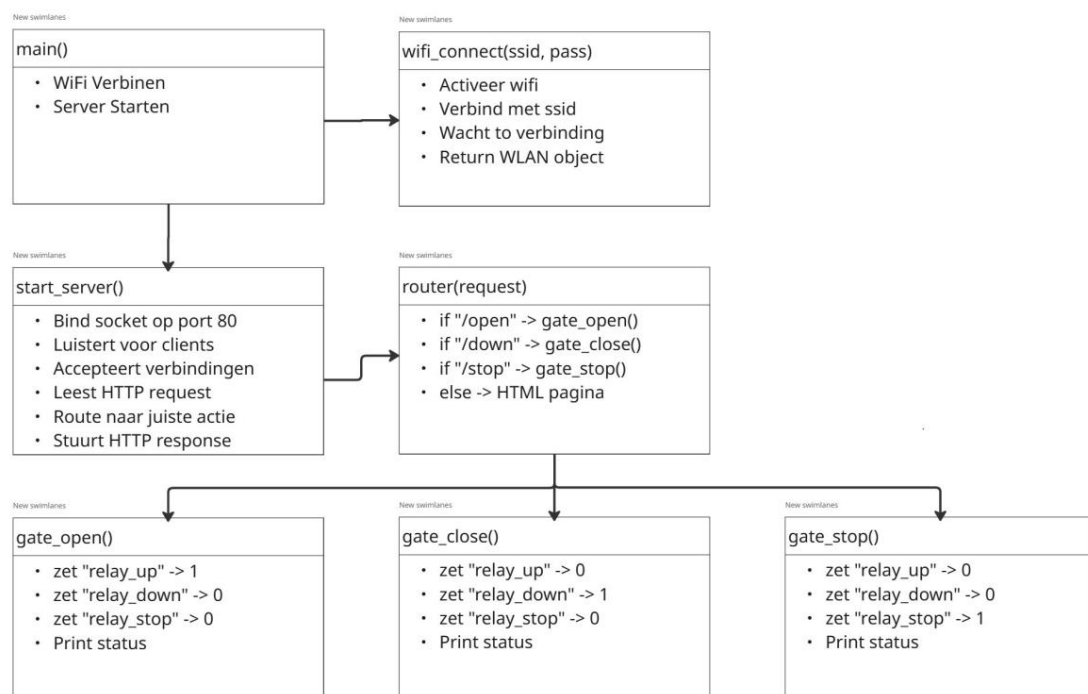
```
42
43     btnDown.addEventListener("click", () => makeRequest("down"));
44
45     function makeRequest(action) {
46         fetch(`http://${window.location.host}/${action}`)
47     }
48
```

```
if 'GET /up' in request:
    gate_up()
    response = make_response('200 OK', 'Gate moved UP')
```

Stabiliteit en foutafhandeling

Ik heb foutafhandeling toegevoegd, zodat het script automatisch stopt wanneer de verbinding met het WiFi mislukt. Ook heb ik try-except blokken toegevoegd voor het afhandelen van clientfouten of socketproblemen. Dit voorkomt dat de hele applicatie crasht.

Functie diagram



Ontwikkeling en Implementatie

De implementatie bestond enerzijds uit het schrijven van de daadwerkelijke software voor het systeem en anderzijds uit het voorbereiden van het systeem voor gebruik in de praktijk. In dit hoofdstuk leg ik eerst kort uit hoe ik de code heb opgebouwd en getest. Daarna geef ik een duidelijke handleiding die beschrijft hoe het systeem geïnstalleerd kan worden op een Raspberry Pi Pico W, zodat het gebruikt kan worden.

Ontwikkeling van de software

De code voor de poortbediening is volledig geschreven in MicroPython. Bij het opzetten van de code heb ik zoveel mogelijk gekozen voor leesbare en herbruikbare functies, zoals `gate_up()`, `gate_stop()` en `gate_down()`. Elke functie schakelt één van de relais 0,5 seconde in. Zo boots ik het gedrag van een fysieke knop na.

Ik heb gewerkt met een logische volgorde:

- Eerst het aansturen van het relais getest via simpele `Pin.value()` command's.
- Vervolgens de WiFi connectie toegevoegd.
- Daarna heb ik de webserver opgezet en uitgebreid met een front-end in HTML en JavaScript.
- Tot slot heb ik foutafhandeling ingebouwd, zodat het systeem stabiel draait, ook bij verbindingsproblemen of fouten in het netwerk.

Testen heb ik gedaan door de code telkens opnieuw te uploaden via de Thonny IDE en het gedrag van het relais te controleren door middel van de ingebouwde led. Ook heb ik het HTML-gedeelte getest op mijn telefoon en laptop, om zeker te weten dat de knoppen correct werken op verschillende browsers en devices.

Voor een diepere toelichting op de code kun je het documentatiebestand lezen dat ik tijdens het coderen heb geschreven. Dit document gaat ook inhoudelijk verder in op de code zelf.

Installatiehandleiding

Met de onderstaande stappen kan iemand het systeem op een Raspberry Pi Pico W installeren en direct gebruiken voor poortbediening.

Benodigdheden

- Raspberry Pi Pico W
- USB-kabel
- Thonny IDE (geïnstalleerd op je pc/laptop)
- Jumper wires en relaismodule (3-kanaals)
- Toegang tot een WiFi netwerk

Stappen

1. Installeer MicroPython op de Pico W
 - Verbind de Pico W via USB met je computer.
 - Druk BOOTSEL in bij het aansluiten en laat dan los.
 - Ga naar de Thonny IDE > Hulpmiddelen > MicroPython installeren op een microcontroller.
 - Kies “Raspberry Pi Pico W” en installeer de firmware.
2. Configureer Thonny
 - Selecteer de juiste poort en kies “MicroPython (Raspberry Pi Pico)” als interpretator.
3. Upload de code
 - Zet het main.py bestand over in de bestanden van de Raspberry Pi
4. Pas WiFi gegevens aan
 - Zorg dat je de juiste `ssid` en `password` instelt in de code.
5. Sluit de relaismodule aan
 - GPIO 6 → Relais 1 (omhoog)
 - GPIO 7 → Relais 2 (stop)
 - GPIO 8 → Relais 3 (omlaag)
 - VBUS → VCC
 - GND → GND
6. Start het systeem
 - Zodra de Pico W opstart, verbindt hij met het WiFi en draait automatisch main.py.
 - Zorg dat je op hetzelfde netwerk zit als de Pico W.
 - Zoek in een browser het lokale IP-adres van de Pico W (bijv. <http://192.168.1.xxx>)
 - Bedien de poort via knoppen op de web interface.

Testfase

Om zeker te zijn dat het prototype van de rolpoortcontroller functioneert zoals bedoeld, ga ik een testfase uitvoeren. In deze fase onderzoek ik of de belangrijkste functies uit de software correct reageren op user-input, of het prototype betrouwbaar is qua verbinding en of de relais module werkt zoals verwacht. De resultaten uit deze tests geven inzicht in de betrouwbaarheid van het prototype en vormen de basis voor verbeteringen richting het eindproduct. Daarnaast stel ik een testplan op voor het uiteindelijke eindproduct. Mocht ik tijdens dit project nog tijd hebben, dan kan ik dit plan invullen. Aangezien de uiteindelijke implementatie buiten de scope van dit project valt, is het echter mogelijk dat dit testplan open blijft zodat ik het later kan invullen.

Testplan en testmethodologieën prototype

Het Testplan richt zich op de belangrijkste functionaliteiten van de software op het prototype:

1. Verbinding maken met het WiFi netwerk
2. Web interface inladen en knoppenbediening
3. HTTP-verwerking door de server (Raspberry Pi)
4. Relaisaansturing (up/stop/down)
5. Stabiliteit bij meerdere gebruikersacties

De testmethode bestaat uit handmatige test met behulp van:

- Een serial monitor voor debug-output (WiFi, request, relaisacties)
- Een browser op desktop en mobiel
- Een multimeter voor het meten van de spanningsverandering op de GPIO-pinnen
- Fysieke observatie van relaisklikken tijdens het testen
- Visuele observatie van de LED-lampjes op relaismodule

Testcases en resultaten

Testcase	Beschrijving	Verwacht resultaat	Resultaat	Opmerking
TC-01	Verbind met WiFi-netwerk	Het systeem maakt binnen 5 seconden verbinding en toont het IP-adres	Succesvol verbonden	Bij geen verbinding wordt correcte timeout-melding weergegeven
TC-02	Web interface opent op IP-adres	HTML-interface wordt correct weergegeven op mobiel en desktop	Laadt goed, responsive layout	Webdesign is altijd goed alleen groot op desktop
TC-03	Klik op knop "UP"	Request naar /up wordt verwerkt, relais 1 schakelt 0.5s in	Werkt goed	Werkt consistent
TC-04	Klik op knop "STOP"	Request naar /stop wordt verwerkt, relais 2 schakelt 0.5s in	Werkt goed	Werkt consistent
TC-05	Klik op knop "DOWN"	Request naar /down wordt verwerkt, relais 3 schakelt 0.5s in	Werkt goed	Werkt consistent
TC-06	Onbekend request (bijv. /random)	HTML-interface wordt als fallback teruggestuurd	Werkt zoals verwacht	Vangnet-functie werkt goed
TC-07	Meerdere verzoeken snel na elkaar	Systeem blijft reageren zonder vastlopen	Server blijft stabiel	Alle verzoeken worden correct verwerkt zonder crash

Bugfixing en kwaliteit

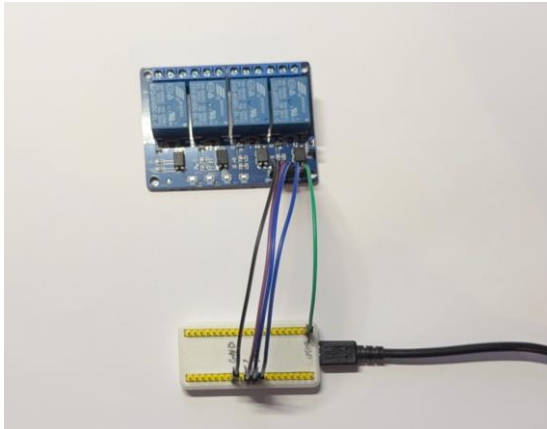
Tijdens de test heb ik geen bugs aangetroffen. Ik heb een paar preventieve maatregelen genomen om de stabiliteit te behouden:

- **Timeout ingebouwd** bij het verbinden met WiFi (RuntimeError na 15 seconden), om te voorkomen dat het systeem eindeloos blijft proberen.
- **Try/Except-structuur** rondom clientverwerking zorgt ervoor dat onverwachte fouten (zoals verbindingsproblemen) niet leiden tot servercrashes.
- **GPIO-reset** na iedere relaisactie zorgt ervoor dat het relais nooit permanent ingeschakeld blijft.

Conclusie

Het prototype werkt betrouwbaar, is stabiel bij gebruik van meerdere functies en vormt een sterke basis voor de uiteindelijke integratie in de fysieke rolpoort. Zowel de hardware (relaisaansturing) als de software (WiFi-connectie, serververwerking en interface) functioneren zoals bedoeld.

Test foto's



Prototype Raspberry Pi Pico W verbonden met een 4 kanaal relais module



Layout screenshot van de layout tijdens het testen

```
MPY: soft reboot
Connecting to WiFi...
Connected. Network config: ('192.168.1.38', '255.255.255.0', '192.168.1.1', '192.168.1.1')
Listening on ('0.0.0.0', 80)
```

```
MPY: soft reboot
Connecting to WiFi...
Traceback (most recent call last):
  File "<stdin>", line 163, in <module>
  File "<stdin>", line 160, in main
  File "<stdin>", line 109, in connect wifi
RuntimeError: WiFi connection timeout

>>>
```

WiFi verbinding serial monitor van de Pi tijdens een goede en slechte verbinding met het WiFi

Testplan en testmethodologieën eindproduct

Deze tweede testfase is gericht op de daadwerkelijke werking van de gekoppelde installatie. De volgende onderdelen zullen in deze fase getest worden:

1. Netwerkverbinding en bereikbaarheid binnen het bedrijfsnetwerk
2. Reactie van de poort op de opdrachten UP / STOP / DOWN
3. Correct gedrag bij herhaalde of snelle opdrachten
4. Gebruiksvriendelijkheid via smartphone (op afstand op het terrein)
5. Veiligheid en overschrijving van het originele systeem
6. Gedrag bij netwerkproblemen of onderbrekingen

De testmethode zal bestaan uit:

- Fysieke interactie met de poort tijdens het bedienen via de web interface
- Monitoring via serial console om te detecteren of er fouten optreden
- Simulatie van netwerkuitval of gebruikersfouten
- Feedback van medewerkers bij eerste gebruik

Testcases

Testcase	Beschrijving	Verwacht resultaat	Resultaat	Opmerking
TC-08	Web interface opent correct in bedrijfsnetwerk	Webpagina is snel en stabiel bereikbaar		
TC-09	UP-command activeert poort naar boven	Poort gaat naar boven		
TC-10	STOP-command stopt de poort op elk moment	Poort stop		
TC-11	DOWN-command activeert poort naar beneden	Poort gaat naar beneden		
TC-12	Opdrachten snel achter elkaar	Systeem blijft stabiel en voert laatste opdracht uit		
TC-13	Bediening via mobiel op terrein	Bediening is mogelijk zonder vertraging		
TC-14	Veiligheid en overschrijving	Poort overschrijft alarm en veiligheid niet		
TC-15	Verbindingsverlies tijdens gebruik	Systeem blijft wachten en doet verder niks		

Conclusie

Hoewel deze tests nog niet zijn uitgevoerd, biedt dit hoofdstuk een volledig voorbereid testplan voor de praktijkfase. Zodra het systeem in gebruik wordt genomen in de echte werkomgeving, kan dit plan direct worden ingezet om de betrouwbaarheid en veiligheid van het systeem te valideren.

Evaluatie en reflectie

Dit project vond ik erg leuk om te doen, vooral omdat het voor mij iets totaal nieuws was. Het combineren van software met fysieke hardware gaf een extra dimensie aan het werk. Het is gewoon tof om iets te programmeren dat daarna ook echt werkt in de praktijk. Dat was voor mij heel anders dan een standaard website maken, en dat maakte het juist zo interessant.

Wat ik jammer vond, is dat ik door de einddatum van mijn stage niet zeker wist of ik het systeem daadwerkelijk kon implementeren op locatie. Daardoor heb ik er bewust voor gekozen om me vooral op het bouwen van een goed werkend prototype te richten. Inmiddels, na het examen, ben ik er wel van overtuigd dat het systeemtechnisch goed werkt en dat ik het zonder veel moeite kan inbouwen. Toch had ik het achter liever al eerder in het project meegenomen.

Tijdens het werken aan dit project heb ik veel geleerd over het gebruik van microcontrollers, zeker de Raspberry Pi Pico W. Maar ook buiten dit project om ben ik gaan kijken naar andere microcontrollers, simpelweg omdat het mijn interesse heeft gewekt. Ook het technische stukje van elektronica, hoewel mijn opzet misschien niet 100% volgens de regels van elektrotechniek is, heb ik beter leren begrijpen. Ik snap nu de basisprincipes, en dat heeft mijn interesse in dit soort projecten alleen maar vergroot.

Een zwak punt van mij is normaal gesproken de voorbereiding en planning van een project. In dit geval heb ik gemerkt dat het veel beter ging doordat ik er bewust mee aan de slag ging. Door vooraf structuur aan te brengen en het project uit te pluizen, had ik een veel duidelijker beeld van wat er moest gebeuren. Dat zorgde voor een efficiënte werkwijze en uiteindelijk voor een beter resultaat. Wel had ik de planning nog strakker mogen bewaken tijdens het proces, maar de basis stond in ieder geval goed.

Kortom: dit project heeft me in korte tijd veel geleerd over een vakgebied waar ik als webdeveloper normaal niet zo snel mee te maken krijg. Het was leerzaam, uitdagend en leuk. Voor mij is dit project zeker geslaagd, en ik zou in de toekomst graag meer van dit soort technische projecten willen doen, al is het maar voor de hobby.

Bijlagen

MicroPython code

De volledige broncode van het project, geschreven in MicroPython. Dus script bestuurt het relais, handelt HTTP-requests af en toont het web interface.

Locatie: Code/main.py

Code notities

Notities en documentatie die ik geschreven heb tijdens het schrijven van de code.

Locatie: Code/Gate Controller Notities.pdf

Functie diagram

Een visuele weergave van de functiestructuur binnen het systeem. Toont hoe de verschillende onderdelen van de code met elkaar samenwerken.

Locatie: Diagrammen/Function diagram.pdf

Projectflow overzicht

Een overzichtelijke flowchart die stap voor stap de werking van het systeem beschrijft: van gebruikersactie tot poortbeweging.

Locatie: Diagrammen/Project flowchart.pdf

Wiring diagram

Schematische tekening van de bedrading en hardware opbouw. Laat zien hoe de Raspberry Pi, relais en poortbediening met elkaar verbonden zijn.

Locatie: Diagrammen/Schematic_rolpoort-automatisering.pdf

Screenshots en foto's project

Verschillende foto's van het prototype en screenshots van de app.

Locatie: Screenshots/

Projectplanning

Gantt-diagram van de projectplanning.

Locatie: planning.xlsx